Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería de Sistemas de Información y Ciencias de la Computación

Plan Fin de Semana – sábado Sede Boca Del Monte

Curso: Programación II

Catedrático: Fernando Javier Caxaj



API REST EN C#

Marli Xiomara Vásquez Pérez

Carnet: 7690-25-10995

Salón: 105 **Sección:** “D”

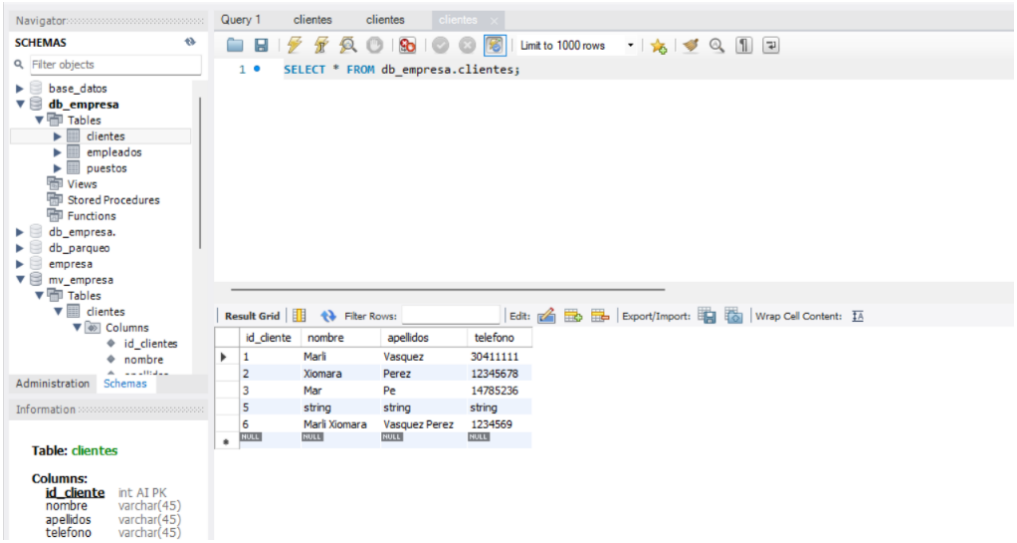
Guatemala, 11 de octubre 2026

DESARROLLO DE UNA WEB API EN .NET CON ENTITY FRAMEWORK CORE, MYSQL Y REACT

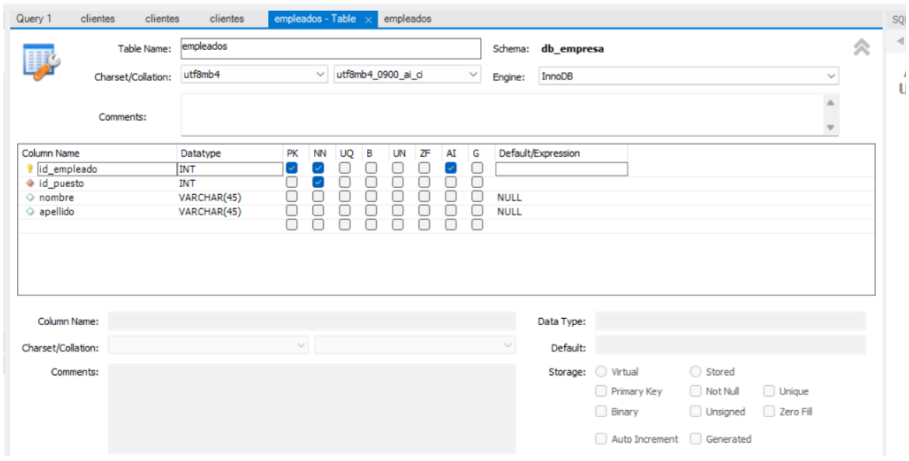
1.1 Creación y preparación de la base de datos

Primero se trabajó con la base de datos MySQL denominada db_empresa. Esta base contenía las tablas clientes, empleados y puestos. Para las pruebas principales de la API se utilizó la tabla clientes, que almacena el identificador del cliente, nombre, apellidos y teléfono.

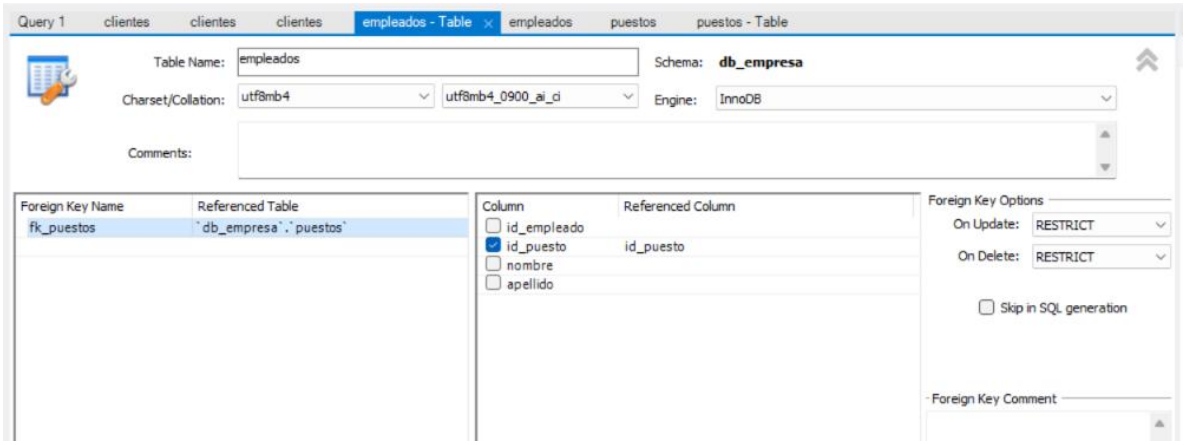
Clientes



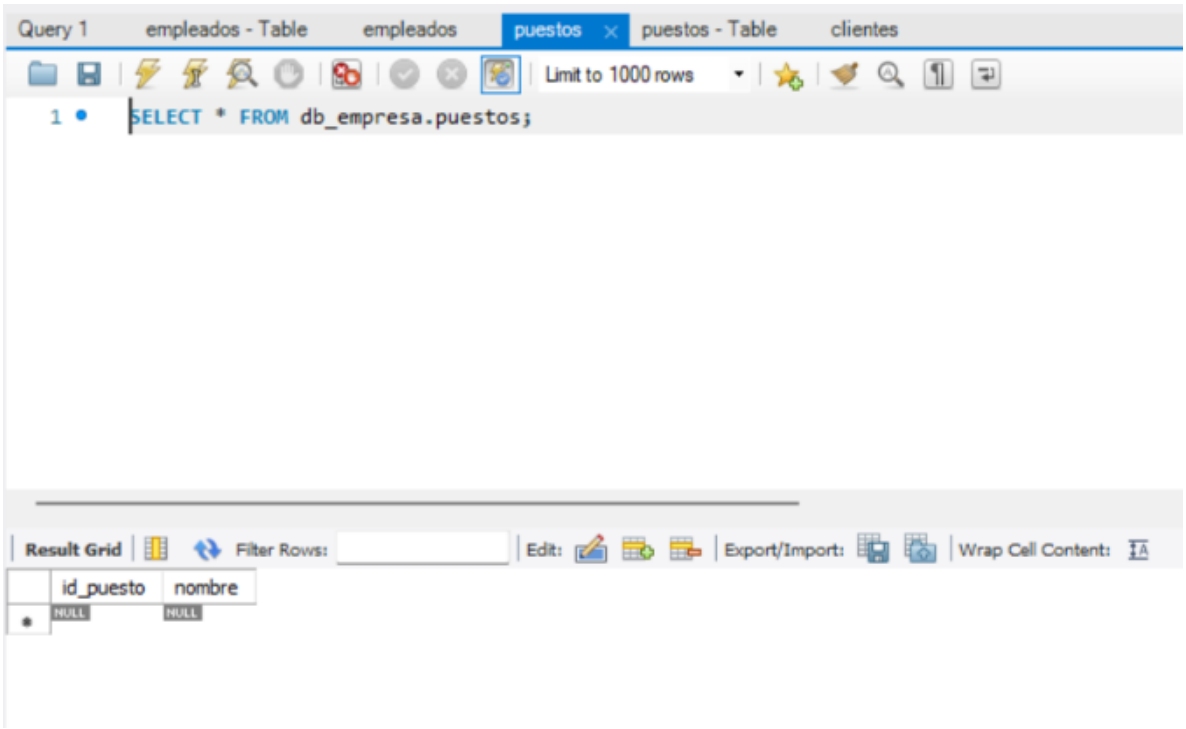
Empleado



Adicional se creó como llave foránea de en empleados el puesto

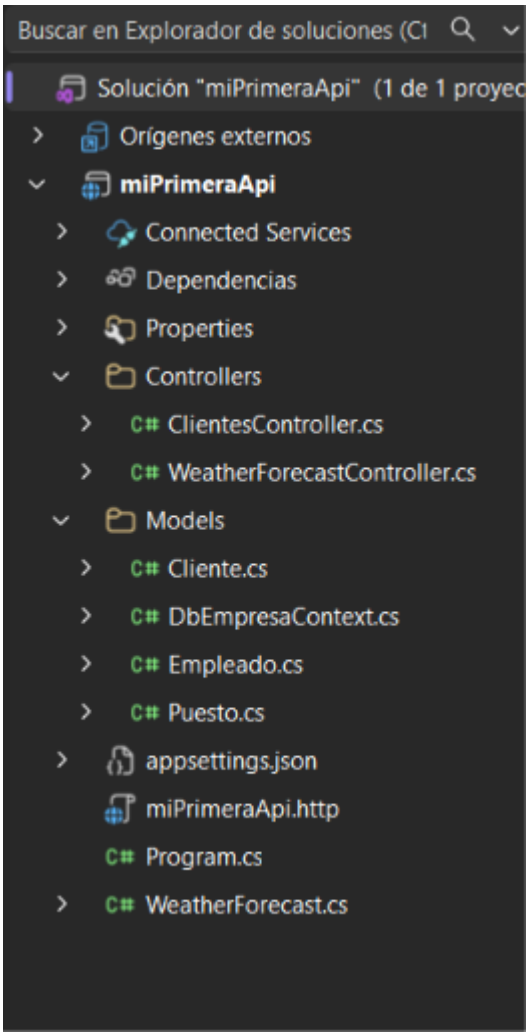


Puesto



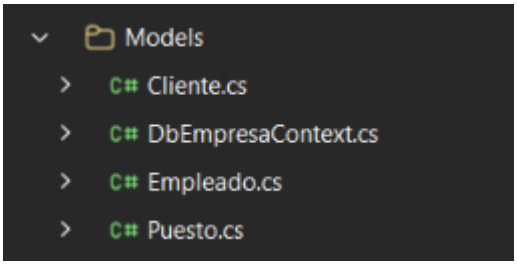
1.2 Creación del proyecto Web API

Se creó el proyecto miPrimeraApi utilizando .NET Web API con controladores. Después se instalaron los paquetes necesarios de Entity Framework Core y el proveedor Pomelo para permitir la conexión entre la aplicación y MySQL.



1.3 Generación de los modelos con Entity Framework Core

A partir de la base de datos existente se utilizó el proceso de scaffolding de Entity Framework Core. Como resultado se generaron las clases Cliente, Empleado y Puesto, además de DbEmpresaContext. Estas clases representan en .NET las tablas y relaciones existentes en MySQL.



1.4 Configuración de la conexión

La conexión a la base de datos se configuró mediante una cadena de conexión y el contexto DbEmpresaContext se registró en Program.cs utilizando UseMySql. De esta forma, los controladores pudieron utilizar Entity Framework Core para consultar y modificar la información almacenada en MySQL.

DbEmpresaContext

```
using System;
using System.Collections.Generic;
using Microsoft.EntityFrameworkCore;
using Pomelo.EntityFrameworkCore.MySql.Scaffolding.Internal;

namespace miPrimeraApi.Models;

public partial class DbEmpresaContext : DbContext
{
    public DbEmpresaContext()
    {
    }

    public DbEmpresaContext(DbContextOptions<DbEmpresaContext> options)
        : base(options)
    {
    }

    public virtual DbSet<Cliente> Clientes { get; set; }

    public virtual DbSet<Empleado> Empleados { get; set; }

    public virtual DbSet<Puesto> Puestos { get; set; }

    //protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
//#warning To protect potentially sensitive information in your connection string, you should move it out of source code. You can avoid scaffolding the connection string by using the Name= syntax to read it from configuration - see https://go.microsoft.com/fwlink/?linkid=2131148. For more guidance on storing connection strings, see https://go.microsoft.com/fwlink/?LinkId=723263.
//                                                
```

```
            .HasColumnName("nombre");
        entity.Property(e => e.Telefono)
            .HasMaxLength(45)
            .HasColumnName("telefono");
    });

    modelBuilder.Entity<Empleado>(entity =>
    {
        entity.HasKey(e => e.IdEmpleado).HasName("PRIMARY");

        entity.ToTable("empleados");

        entity.HasIndex(e => e.IdPuesto, "fk_puestos_idx");

        entity.Property(e => e.IdEmpleado).HasColumnName("id_empleado");
        entity.Property(e => e.Apellido)
            .HasMaxLength(45)
            .HasColumnName("apellido");
        entity.Property(e => e.IdPuesto).HasColumnName("id_puesto");
        entity.Property(e => e.Nombre)
            .HasMaxLength(45)
            .HasColumnName("nombre");

        entity.HasOne(d => d.IdPuestoNavigation).WithMany(p => p.Empleados)
            .HasForeignKey(d => d.IdPuesto)
            .OnDelete(DeleteBehavior.ClientSetNull)
            .HasConstraintName("fk_puestos");
    });

    modelBuilder.Entity<Puesto>(entity =>
    {
        entity.HasKey(e => e.IdPuesto).HasName("PRIMARY");

        entity.ToTable("puestos");

        entity.Property(e => e.IdPuesto).HasColumnName("id_puesto");
        entity.Property(e => e.Nombre)
            .HasMaxLength(45)
            .HasColumnName("nombre");
    });

    OnModelCreatingPartial(modelBuilder);
  }

  partial void OnModelCreatingPartial(ModelBuilder modelBuilder);
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using miPrimeraApi.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<DbEmpresaContext>(options =>
```

```
{
    options.UseMySql(
        builder.Configuration.GetConnectionString("conexion"),
        ServerVersion.AutoDetect(builder.Configuration.GetConnectionString("conexion"))
    );
});

builder.Services.AddCors(options =>
{
    options.AddPolicy("NuevaPolitica", app =>
    {
        app.AllowAnyOrigin()
           .AllowAnyHeader()
           .AllowAnyMethod();
    });
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseCors("NuevaPolitica");

app.UseAuthorization();

app.MapControllers();

app.Run();
```

1.5 Creación del controlador de clientes

Se creó ClientesController para concentrar las operaciones relacionadas con la tabla clientes. El controlador recibe DbEmpresaContext mediante su constructor y utiliza el contexto para acceder a los registros.

ClientesController

```
using Microsoft.AspNetCore.Mvc;
using miPrimeraApi.Models;

namespace miPrimeraApi.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class ClientesController : Controller
    {
        private readonly DbEmpresaContext _context;
        public ClientesController(DbEmpresaContext ctx)
        {
            _context = ctx;
        }
```

```
        [HttpGet("ListarUsuarios")]
        public ActionResult<List<Cliente>> ListarUsuarios()
        {
            return _context.Clientes.ToList();
        }

        [HttpGet("ObtenerUsuario/{id}")]
        public ActionResult<Cliente> ObtenerUsuario(int id)
        {
            return _context.Clientes.FirstOrDefault(e => e.IdCliente == id);
        }

        [HttpPost("CrearUsuario")]
        public ActionResult<Cliente> CrearUsuario(Cliente cliente)
        {
            _context.Clientes.Add(cliente);
            _context.SaveChanges();

            return cliente;
        }

        [HttpPut("ActualizarUsuario/{id}")]
        public ActionResult<Cliente> ActualizarUsuario(int id, Cliente cliente)
        {
            var usuario = _context.Clientes.FirstOrDefault(e => e.IdCliente == id);

            usuario.Nombre = cliente.Nombre;
            usuario.Apellidos = cliente.Apellidos;
            usuario.Telefono = cliente.Telefono;

            _context.SaveChanges();

            return usuario;
        }

        [HttpDelete("EliminarUsuario/{id}")]
        public ActionResult<Cliente> EliminarUsuario(int id)
        {
            var usuario = _context.Clientes.FirstOrDefault(e => e.IdCliente == id);

            _context.Clientes.Remove(usuario);
            _context.SaveChanges();

            return usuario;
        }
    }
}
```

1.6 Implementación de las operaciones CRUD

Método HTTP	Endpoint	Función
GET	/Clientes/ListarUsuarios	Obtiene la lista completa de clientes.
GET	/Clientes/ObtenerUsuario/{id}	Busca un cliente por su identificador.

POST	/Clientes/CrearUsuario	Agrega un nuevo cliente a la base de datos.
PUT	/Clientes/ActualizarUsuario/{id}	Modifica los datos de un cliente existente.
DELETE	/Clientes/EliminarUsuario/{id}	Elimina un cliente de la base de datos.

Con estas operaciones se completó el CRUD del módulo de clientes. GET permitió realizar consultas, POST insertar información, PUT actualizar registros y DELETE eliminar registros.

```
[HttpPost("CrearUsuario")]
public ActionResult<Cliente> CrearUsuario(Cliente cliente)
{
    _context.Clientes.Add(cliente);
    _context.SaveChanges();

    return cliente;
}

[HttpPut("ActualizarUsuario/{id}")]
public ActionResult<Cliente> ActualizarUsuario(int id, Cliente cliente)
{
    var usuario = _context.Clientes.FirstOrDefault(e => e.IdCliente == id);

    usuario.Nombre = cliente.Nombre;
    usuario.Apellidos = cliente.Apellidos;
    usuario.Telefono = cliente.Telefono;

    _context.SaveChanges();

    return usuario;
}

[HttpDelete("EliminarUsuario/{id}")]
public ActionResult<Cliente> EliminarUsuario(int id)
{
    var usuario = _context.Clientes.FirstOrDefault(e => e.IdCliente == id);

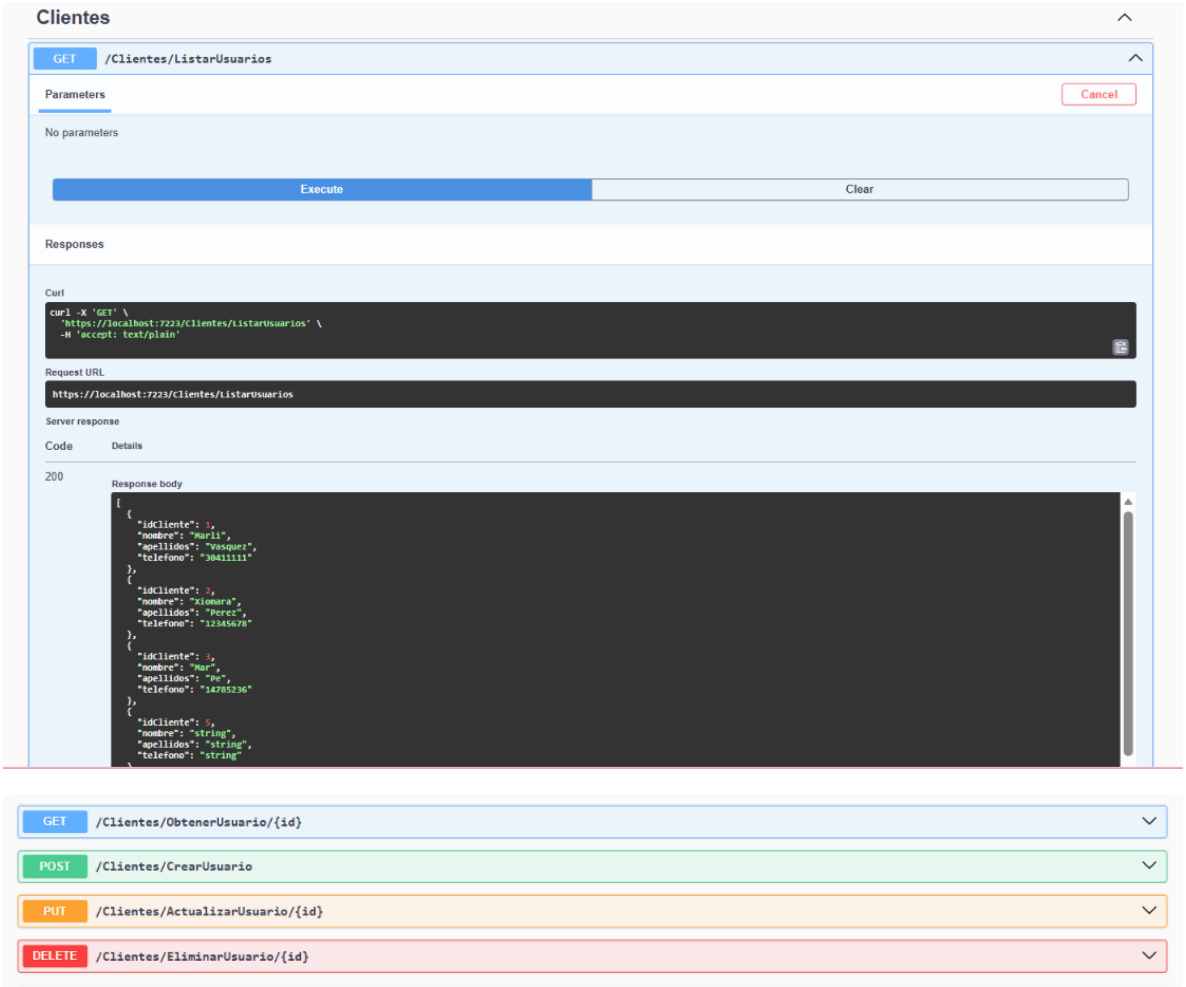
    _context.Clientes.Remove(usuario);
    _context.SaveChanges();

    return usuario;
}
```

1.7 Pruebas mediante Swagger

Después de implementar el controlador, se ejecutó el proyecto y se utilizó Swagger para probar cada endpoint. Se verificó que la consulta mostrara los clientes existentes y que las

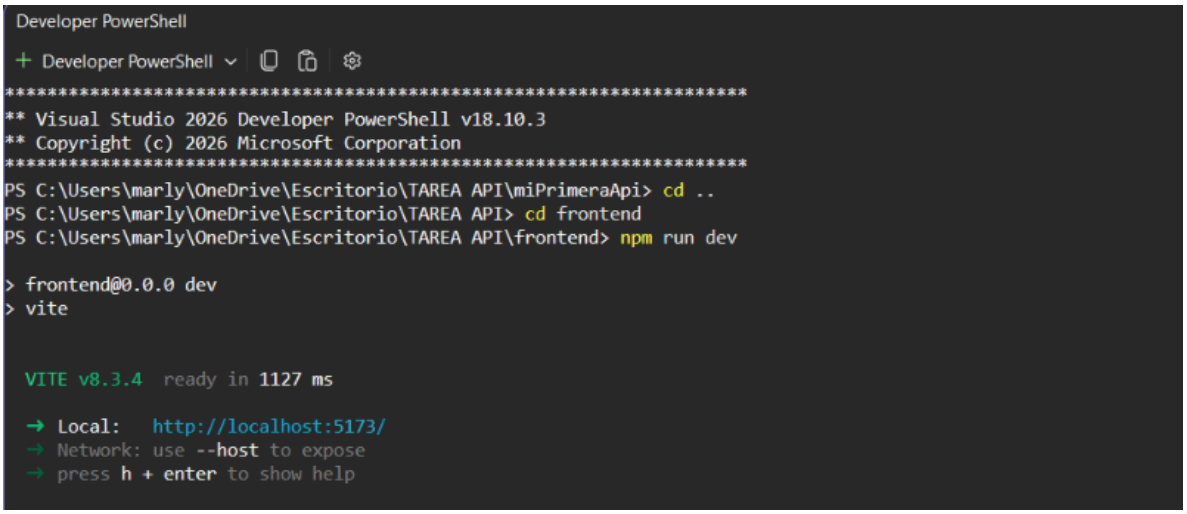
operaciones de creación, actualización y eliminación realizaran correctamente los cambios en MySQL.



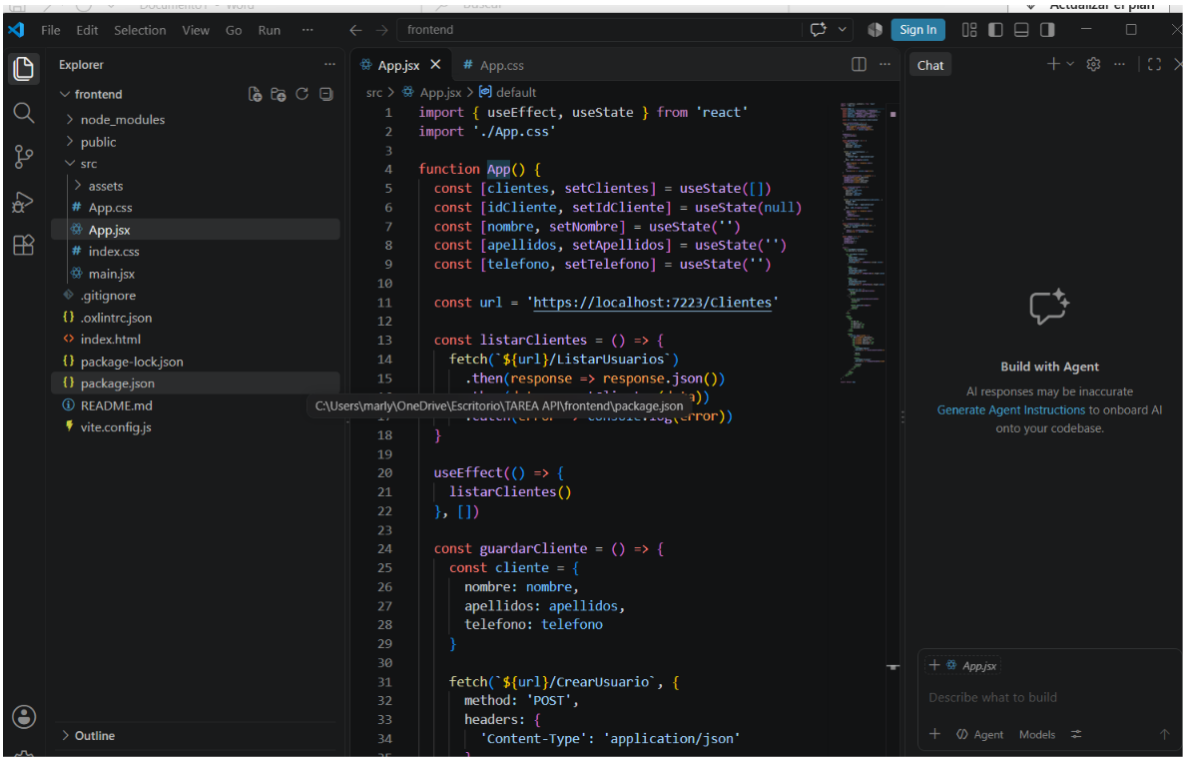
1.8 Desarrollo de la conexión con React

Como complemento del proyecto se instaló Node.js y se creó un frontend con React mediante Vite. La interfaz se configuró para realizar peticiones HTTP hacia los endpoints de ClientesController. También se habilitó CORS en la Web API para permitir la comunicación entre el frontend ejecutado en localhost y la API.

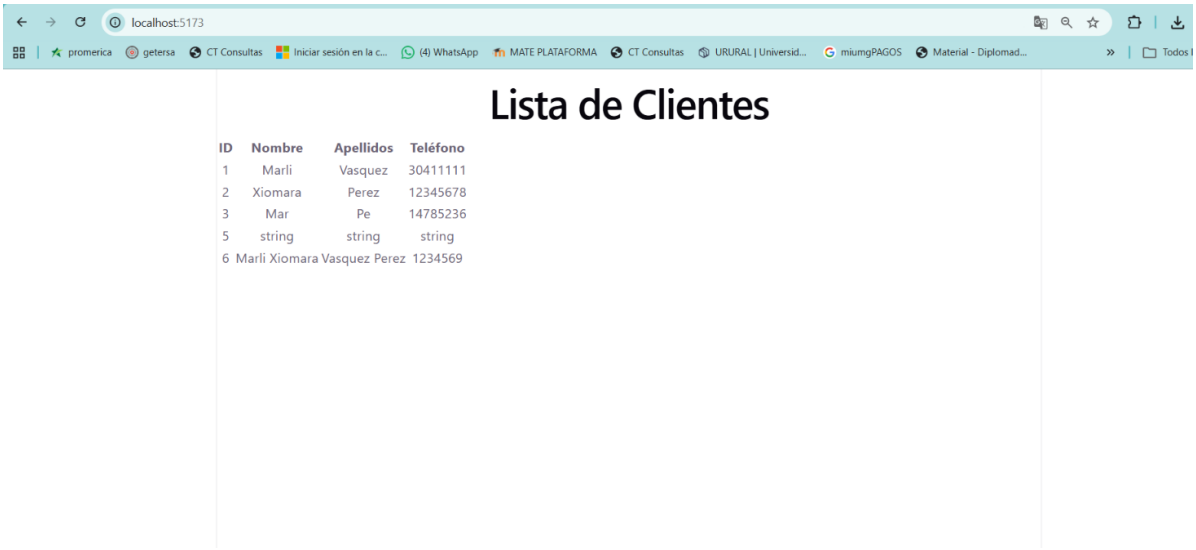
Teminal en Visual studio 2026



Visual Studio code



Local Host 5173 conexión React



1.9 Interfaz para administrar clientes

En React se creó una pantalla denominada Gestión de Clientes. La aplicación obtiene la lista de clientes mediante el endpoint ListarUsuarios y presenta los resultados en una tabla. Además, se incorporó un formulario para ingresar nombre, apellidos y teléfono, junto con las opciones para agregar, editar, actualizar, cancelar y eliminar registros.

